SARCache: Controlling Forecast Age in Diffusion Transformers

Ben Gao[1,2] Situo Zhang[6] Yanchao Li[1,3] Di Zhang[1,4] Jiaqing Xie[1,4]
Penghui Li[2,5] Yeyun Chen[1] Tianfan Fu[1,3] Shijie Wang[1] Yuqiang Li[1]

[1]Shanghai Artificial Intelligence Laboratory, Shanghai, China
[2]Institute for Advanced Studies, Wuhan University, Wuhan, China
[3]School of Computer Science, Nanjing University, Nanjing, China
[4]School of Computer Science, Fudan University, Shanghai, China
[5]Shanghai Innovation Institute, Shanghai, China
[6]X-LANCE Lab, School of Computer Science,
Shanghai Jiao Tong University, Shanghai, China

Abstract

Reducing inference cost is important for making large diffusion transformers practical, especially when faster generation must preserve the structure and details of the original output. Caching saves computation by reusing intermediate results. Predicting how those results change can improve reuse, but a recent trend may become unreliable before the cache needs to be refreshed. We introduce Staleness-Aware Residual Caching (SARCache), which separates how long to predict from when to recompute. The method predicts the change across the transformer stack for a limited number of steps, then reuses the latest observed change while updating the input projection and output head. With the same 13 full evaluations on Qwen-Image-2.1, a four-step prediction window reduces mean LPIPS by 29.2% relative to unbounded prediction and 15.1% relative to a predictor that holds its final forecast. All six paired comparisons across two confirmation schedules favor this policy. System measurements show useful speed–fidelity trade-offs, including 1.85× acceleration at LPIPS 0.0126. Transfer experiments show that the preferred window depends on the model and schedule. These findings support controlling prediction duration separately from cache refresh timing.
Project blog: `flyben97.github.io/SARCache-blog/`

1 Introduction

Diffusion transformers generate images through repeated evaluations of a large network. Diffusion and score-based models gradually remove noise [1, 2]. Latent diffusion reduces computation by working in a compact image representation [3], while transformer backbones and flow matching support scalable generation [4–6]. Neighboring steps often perform similar work. Reusing that work can reduce inference time, but the resulting image should still preserve the composition and details of the original output for the same prompt and seed.

A cache can hold a previous result or predict how it will change. Holding is simple but ignores change. Prediction follows recent changes but assumes that they will continue. This assumption can become less accurate as more steps pass without recomputation. An earlier observation may therefore remain useful after its recent trend stops being a reliable guide.

Existing methods address both reuse timing and feature prediction. TeaCache, EasyCache and SeaCache use inexpensive signals to decide when to recompute [7–9]. TaylorSeer predicts skipped features [10], while confidence gating, DiCache and RFC also use current features to assess or improve predictions [11–13]. We focus on the duration of prediction within a reuse interval. Stopping a prediction need not end reuse or trigger another full transformer evaluation.

Staleness-Aware Residual Caching (SARCache) makes this choice explicit. It stores the difference between the input embedding and the final image-token representation, called the residual across the transformer stack. At a reuse step, it adds an estimated residual to the current embedding and applies the current output head. It predicts this residual only within a configured window. After the window, it returns to the latest observed residual and keeps skipping the transformer until the controller requests a refresh.

We make three contributions. First, we introduce a prediction rule that returns to the latest observation after a finite window, independently of refresh timing. Second, we characterize when this return reduces local residual error and test it against holding the final forecast at identical refresh steps. Third, we measure speed and fidelity across schedules and models. Qwen results support prediction followed by holding. Transfer and video results identify settings that favor other policies.

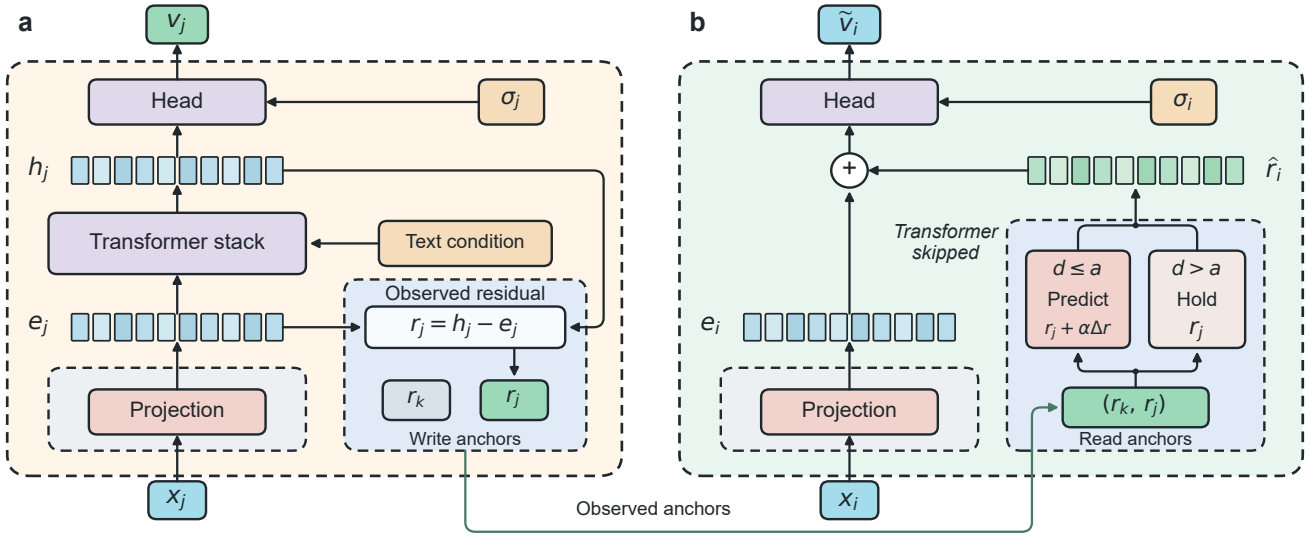


Figure 1. Full refresh and residual reuse with a finite prediction window. a, A full evaluation stores the residual across the transformer stack $r_j = h_j - e_j$ and updates the observed anchors. **b,** Reuse skips the transformer but recomputes the input projection and head conditioned on the noise level. With two anchors, SARCache predicts $r_j + \alpha\Delta r$ while the age $d = i - j \le a$, where $\alpha = d/(j-k)$ and $\Delta r = r_j - r_k$. After the horizon, or with only one anchor, it holds r_j. Only full evaluations update the anchors. The gate and reuse cap control refresh timing separately. The green connector carries stored anchors. Token strips are schematic and do not encode measured activations or tensor dimensions.

2 Related Work

What to cache. Block Caching reuses outputs of network blocks [14]. DeepCache and Faster Diffusion reuse U-Net features that change slowly between steps [15, 16]. For transformers, FORA caches attention and MLP outputs [17], Pyramid Attention Broadcast assigns reuse intervals to attention operations [18], and ToCa selects tokens and layers for reuse [19]. FasterCache also uses redundancy between guidance branches [20]. Δ-DiT retains current-input information through cached differences across block groups [21]. SARCache stores the change across the whole transformer stack and recomputes the input projection and output head.

When to refresh. TeaCache uses a cheap feature probe, EasyCache estimates how changes propagate, MagCache models changes in residual magnitude, and First Block Cache uses early-block information [7, 8, 22, 23]. SeaCache filters the refresh signal to separate content changes from noise [9]. Our image controller follows this principle. Confidence-Gated Taylor checks first-block prediction error and requests full computation when prediction is unreliable [11]. DiCache uses shallow-layer probes to choose reuse steps and combine cached features [12]. RFC uses input–output relationships to improve prediction and estimate when recomputation is needed [13]. These works address both prediction and scheduling. Our tests with fixed refresh steps keep the schedule identical across policies.

How to predict. TaylorSeer extrapolates features with Taylor expansions [10]. FoCa treats feature evolution as an ODE and corrects forecasts using recent observations [24]. HiCache uses scaled Hermite polynomials [25], D2Cache uses second-order changes in model predictions [26], and L2P learns timestep-specific linear weights from data [27]. HyCa assigns different prediction rules to feature dimensions with different dynamics [28]. The Diffusers SeaCache hook also supports residual prediction and a reuse limit [29]. We study a specific choice within this broader design space: stopping extrapolation after a fixed window and returning to the latest observed residual while reuse continues. Our Clipped control uses the same prediction window to test the effect of returning to the observation.

Separating control decisions. Recent Frequency-Step Cache separates scheduled full synchronization from intermediate repair in autoregressive video diffusion [30]. Its repair can recompute selected tokens or full steps. SARCache separates refresh timing from the duration of residual extrapolation. Ending extrapolation switches to residual holding without requesting a full evaluation. This distinction matters when comparing the mechanisms and their computational costs.

Other routes to faster generation. DDIM, EDM, DPM-Solver and UniPC improve sampling [31–35], while progressive distillation and consistency models reduce the number of model evaluations through training [36–38]. DistriFusion and AsyncDiff use cached information to support parallel execution [39, 40]. Our experiments retain the image sampler and model and measure agreement with the uncached output for the same prompt and seed, following SeaCache [9]. Earlier DrawBench sweeps [41] informed development. The main experiments separate calibration from test prompts.

3 Method

3.1 Reusing the residual, updating the current state

SARCache skips the expensive transformer stack but still computes the input projection and output head (Fig. 1). Let x_i be the image latent (the current image representation) at step i, $e_i = E_\theta(x_i)$ its embedding, and h_i the final image-token representation. A full evaluation stores the residual and computes the model output:

$$r_i = h_i - e_i, \qquad v_i = H_\theta(e_i + r_i, \sigma_i), \tag{1}$$

where H_θ is the output head conditioned on the timestep and σ_i is the noise level. Text conditioning is implicit in h_i. The residual is the change that the transformer stack makes to its input embedding.

At a reuse step, the stack is replaced by an estimated residual $\widehat{r}_i$:

$$\widetilde{v}_i = H_\theta\big(E_\theta(x_i) + \widehat{r}_i, \sigma_i\big). \tag{2}$$

Holding the residual does not freeze the output: the projection and head still use the current latent and noise level. For the image models, this approximation enters the original Euler update,

$$x_{i+1} = x_i + (\sigma_{i+1} - \sigma_i)\widetilde{v}_i. \tag{3}$$

The sampling steps remain unchanged. The video adapter retains its native multistep solver.

3.2 Limiting the forecast horizon

The two latest full evaluations show how the residual has recently changed. Let $k < j < i$ denote those two refresh steps and the current step. Linear prediction extends the change $r_j - r_k$ according to the number of elapsed steps. SARCache permits it only within a prediction window (or horizon) a:

$$\widehat{r}_i = \begin{cases} r_j + \dfrac{i-j}{j-k}(r_j - r_k), & i - j \le a,\ k \text{ exists}, \\ r_j, & \text{otherwise.} \end{cases} \tag{4}$$

Forecast age counts the steps since the latest full evaluation. Once it exceeds a, the method uses the observed residual r_j again. The two stored observations, called anchors, are updated only by full evaluations. Predicted values never replace them.

We evaluate $a \in \{0, 1, 2, 3, 4, \infty\}$. Setting $a = 0$ gives holding, and $a = \infty$ gives unbounded linear prediction. The horizon is fixed for each configuration. Its effect depends on the allowed reuse interval: if a positive reuse cap q satisfies $q \le a$, a refresh occurs before the return to holding can take effect.

3.3 Choosing when to refresh

The prediction window controls how the cache is used. The controller decides when to recompute and replace it. Following SeaCache [9], the image controller measures change in a spectrally filtered latent, $z_i = \mathcal{W}_{\sigma_i}(x_i)$:

$$d_i = \frac{\langle |z_i - z_{i-1}| \rangle}{\langle |z_{i-1}| \rangle + \epsilon}, \tag{5}$$

where $\langle |\cdot| \rangle$ is the mean absolute value and $\epsilon = 10^{-8}$. The filter acts only on this indicator. The residual used for reconstruction is unchanged.

The controller accumulates d_i and requests a full evaluation when the sum exceeds a threshold δ. A refresh resets this sum. The first w steps use full evaluations to initialize the history. An empty cache, the final step, or q consecutive reuse steps also forces a refresh. The setting $q = 0$ disables the cap. The indicator is updated at every step, including reuse steps. An alternative gate estimates propagation from changes in the embedding and output [8].

The gate estimates change from the latent indicator. A refresh restores an exact model evaluation at the current state, with previous approximation errors carried forward by the sampler. Our controlled experiments use identical refresh steps to isolate the effect of prediction.

3.4 When should prediction stop?

We analyze the residual estimates at a common current state and anchor history. Let $d = i - j$, $v = (r_j - r_k)/(j - k)$, and $u = r_i^\star - r_j$, where $r_i^\star$ is the residual from a full evaluation at that state. The estimates are $r_j + bv$, with $b = 0$ for holding, $b = d$ for Unbounded, and $b = a$ for Clipped after the window.

Proposition 1 (criterion for returning to the observation). For any $b > 0$, the difference in squared Euclidean residual error is

$$\|u - bv\|_2^2 - \|u\|_2^2 = b^2\|v\|_2^2 - 2b\langle u, v\rangle. \tag{6}$$

Thus, at $d > a > 0$, Finite has no larger local error than both Clipped and Unbounded if $2\langle u, v\rangle \le a\|v\|_2^2$. This follows by expanding the squared norm and using $d > a$. The condition measures how much of the historical trend persists. A continuing linear trend favors prediction. A small or reversed displacement favors returning to the observation.

Corollary 1 (uncertainty after the window). If the available information is only $\|u\|_2 \le R$, then for $b \ge 0$,

$$\sup_{\|u\|_2 \le R} \|u - bv\|_2 = R + b\|v\|_2. \tag{7}$$

The triangle inequality gives the upper bound, attained by $u = -Rv/\|v\|_2$ for $v \neq 0$. Holding minimizes this worst-case error. This uncertainty model motivates discarding an old trend when its direction becomes unreliable. It leaves the useful duration of that trend to empirical selection.

Proposition 2 (when the window is inactive). With identical initial states, deterministic computation and a fixed refresh schedule, Finite, Clipped and Unbounded generate identical trajectories if every reuse age satisfies $d \leq a$. At each such step their estimates coincide, and induction preserves equal states and anchors. The same result holds with an identical deterministic controller and a positive reuse cap $q \leq a$. Consequently, testing the return to holding requires reuse steps beyond a.

These statements concern local residual geometry and policy equivalence. The output head and subsequent sampling steps also affect image fidelity. Section 4.3 tests that fidelity directly. A Lipschitz head bounds output error by its Lipschitz constant times residual error, with later Euler updates propagating that error along the trajectory.

3.5 Image and audio–video adapters

The image adapters reconstruct the image-token rows and use the current output head. Evaluations use a single denoising branch: the guidance-1 setting for Qwen and guidance-0 convention for Krea. These values follow each model's API. Experiments with multiple guidance branches are outside our scope.

For H3, the residual covers the packed audio–video representation. Reuse reconstructs this representation before the native video and audio output operations, retaining solver history and audio conversion. SARCache-H3 and Sea-H3 share a gate based on first-block modulated features. A three-dimensional SEA filter processes video time and space. The audio probe remains unfiltered. Separate drift accumulators trigger a joint refresh when either exceeds the threshold. This adapter applies the residual-caching principle with a representation and gate suited to joint generation.

4 Experiments

4.1 Setup

We evaluate system speed and fidelity, then compare prediction rules at identical refresh steps. The image models are Qwen-Image-2.1 (Qwen) and Krea-2 (Krea), with suffixes 1024 and 2048 indicating resolution. MiniMax-H3 provides an exploratory audio–video evaluation [42–45].

Data and experimental design. The original image evaluation selects configurations on calibration prompts and fixes them before testing. Qwen-1024 uses 200 test prompts with three seeds. Qwen-2048 and Krea each use 100 prompts with three seeds. Image prompts are custom subsets of PartiPrompts, GenEval and authored examples. Video prompts draw on VBench sources and authored examples [46–48].

A subsequent Qwen-1024 study measures five operating points for each of SARCache, SeaCache, TeaCache, EasyCache and a second-order Taylor output predictor. Its 15,000 candidate runs reuse the original 200 test prompts and three seeds. Thresholds are selected on calibration subsets of 8 and 24 prompts. SARCache uses warmup $w = 8$, reuse cap $q = 6$ and horizon $a = 3$. Taylor varies its number of full evaluations on fixed schedules. Models, samplers and bf16 precision on H200 are retained. This study extends the original evaluation on an already studied test set.

For the controlled prediction experiment, 80 development prompts select the horizons. A separate set of 80 confirmation prompts, with seeds 0/1, tests the selected policies without retuning. Two confirmation prompts appeared in an earlier development campaign. We retain all 80 in the primary analysis and report a sensitivity analysis on the remaining 78.

Metrics and uncertainty. We compare native-resolution RGB images with the uncached output for the same prompt and seed. Lower LPIPS [49], higher SSIM [50] and higher PSNR measure closer reference agreement. Prompt adherence and human preference require separate evaluation. Request time includes preparation, denoising and decoding, and excludes model loading, media writing and evaluation. Speedup is the ratio of mean reference time to mean method time within each runner.

We estimate uncertainty with 10,000 bootstrap resamples of prompts, keeping each prompt's seeds together. Holm correction covers the six LPIPS comparisons. Confirmation statistics use the complete paired export at five decimal places.

4.2 System speed and reference fidelity

Performance across models. Table 1 shows that the benefit depends on the model and operating point. SARCache has substantially lower LPIPS than SeaCache at Qwen-1024 A, with modestly longer runtime. At B, their times are close and SeaCache has lower LPIPS. Qwen-2048 A trades shorter runtime for slightly higher LPIPS, while Krea B favors SeaCache in LPIPS. The comparison therefore calls for examining several measured operating points.

Five operating points per method. Figure 2 identifies a useful middle operating region for SARCache. It reaches LPIPS 0.0126 at 1.85× speedup, with lower LPIPS than EasyCache at nearly the same runtime. SeaCache offers a nearby, faster point with slightly higher LPIPS. At another pair of measured points, SARCache is 5.4% faster than SeaCache and improves PSNR by 0.253 dB, while LPIPS favors SeaCache. Pixel and perceptual metrics thus capture different aspects of reference fidelity.

Under heavier acceleration, SARCache's shorter runtime comes with a larger LPIPS penalty than EasyCache. TeaCache and the evaluated Taylor output adaptation have higher LPIPS over this grid. The Taylor results concern the specified output predictor. Cal-

Table 1. Frozen image configurations on held-out prompts. Means over 200 prompts with three seeds for Qwen-1024 and 100 prompts with three seeds for each other setting. Time is measured end to end in seconds. PSNR uses RGB. A/B group configurations by calibration target. Bold marks the best displayed fidelity mean within each model and group.

		Qwen-1024			Qwen-2048			Krea-1024		
Method	Band	Time↓	LPIPS↓	PSNR↑	Time↓	LPIPS↓	PSNR↑	Time↓	LPIPS↓	PSNR↑
SARCache	A	3.529	**0.0115**	**38.85**	17.932	0.0261	36.20	8.288	0.0492	33.71
SeaCache		3.292	0.0225	35.33	18.793	**0.0247**	**37.33**	8.786	**0.0486**	**33.72**
TeaCache		3.128	0.2161	20.56	19.256	0.1756	22.19	8.650	0.1176	28.66
EasyCache		3.265	0.0235	34.25	18.309	0.0275	34.57	8.813	0.0752	31.65
SARCache	B	2.488	0.0496	**31.33**	15.277	0.0693	**32.00**	6.943	0.0570	32.95
SeaCache		2.517	**0.0435**	31.23	14.259	0.0665	31.36	7.086	**0.0455**	**34.68**
TeaCache		2.414	0.2926	18.47	13.563	0.2550	19.81	7.054	0.1624	26.41
EasyCache		2.430	0.0544	30.60	14.008	**0.0659**	30.93	6.769	0.3481	26.58

Full-reference times: 6.303, 36.853, and 17.647 s, respectively. Calibration bands were missed by EasyCache (Krea A), TeaCache and EasyCache (Qwen-2048 B), and TeaCache (Krea B). SARCache Qwen-1024 A exceeded its confirmation target by 6.4%. Comparisons use the measured test times above.

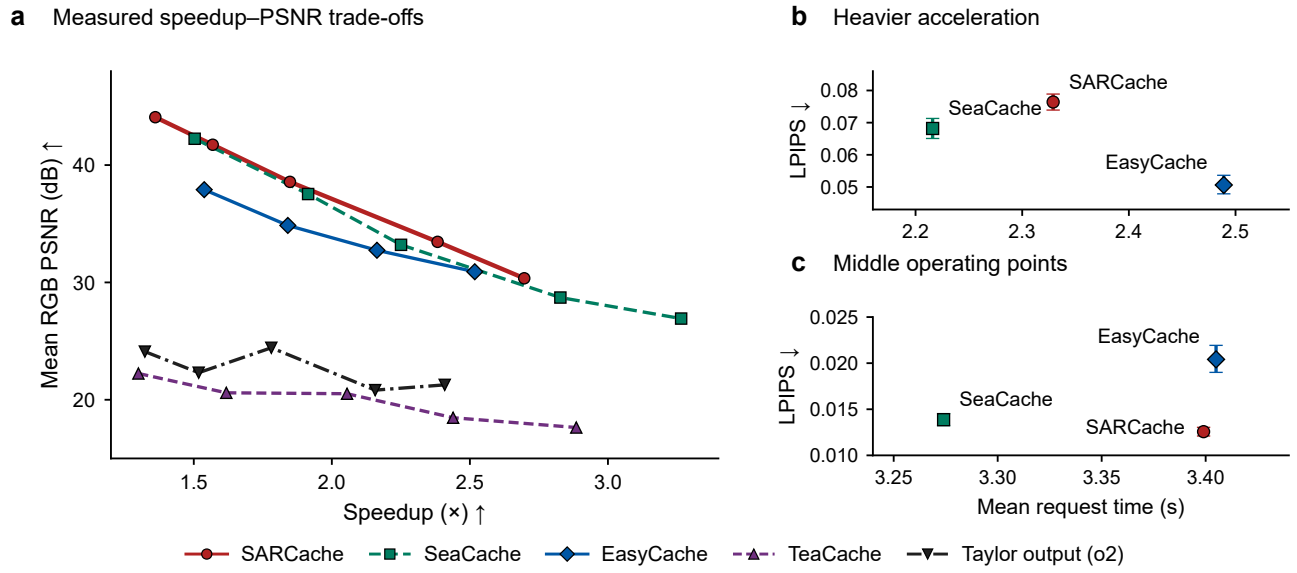


Figure 2. Qwen-Image-2.1 speed and fidelity at 1024 resolution. a, Mean RGB PSNR versus measured speedup over each runner's uncached baseline for five methods, showing the 24 configurations with finite PSNR in the 25-point study. Each uses 200 prompts and three seeds. Lines connect measured points as visual guides. PSNR intervals are unavailable. **b,c,** Local views of heavier and middle operating points, showing mean LPIPS against measured request time. Vertical bars show the supplied 95% LPIPS intervals. Each panel uses its own axis ranges.

ibration targets are missed by all five SeaCache configurations, and test timing falls outside the allowed time-ratio tolerance for five SeaCache and two TeaCache points. All points are retained at their measured times. The study has no finite CLIP scores. Its $q = 6 > a = 3$ setting permits a return to holding, with the activation frequency unavailable from the retained records.

Preserving local structure. Figure 4 illustrates differences around the bus windscreen. SARCache preserves the reference border, wiper and visible interior closely. EasyCache changes the window contour, SeaCache changes the framing and interior, and TeaCache alters the overall geometry. The case was selected for favorable SARCache differences against SeaCache and TeaCache. Exact comparator configurations and individual runtimes are unavailable, so it illustrates local structure at the supplied settings.

4.3 The effect of limiting prediction age

Comparison at identical refresh steps. We compare four policies while fixing every full evaluation. **Hold** reuses the latest observed residual. **Unbounded** continues linear extrapolation. **Finite**, our policy, predicts within a window and then returns to the observation. **Clipped** uses the same window but keeps its final forecast, replacing $i - j$ with $\min(i - j, a)$. Finite and Clipped differ only after the window. Each policy evolves its own states and updates anchors at the

Table 2. Qwen confirmation at identical refresh positions. LPIPS difference (Finite minus comparator), with negative values favoring Finite. Intervals are 95% prompt-cluster bootstrap intervals, and p_H applies Holm adjustment across all six tests. Each mask has 80 prompts and two seeds. Statistics use the complete paired export at five decimal places.

	$g4$: $a = 3$, 16 full calls			$g8$: $a = 4$, 13 full calls		
Comparator	Δ	95% CI	p_H	Δ	95% CI	p_H
Hold	-0.0099	[-0.0116, -0.0085]	0.0006	-0.0147	[-0.0161, -0.0134]	0.0006
Unbounded	-0.0027	[-0.0037, -0.0017]	0.0006	-0.0307	[-0.0334, -0.0279]	0.0006
Clipped	-0.0012	[-0.0020, -0.0004]	0.0031	-0.0132	[-0.0149, -0.0114]	0.0006

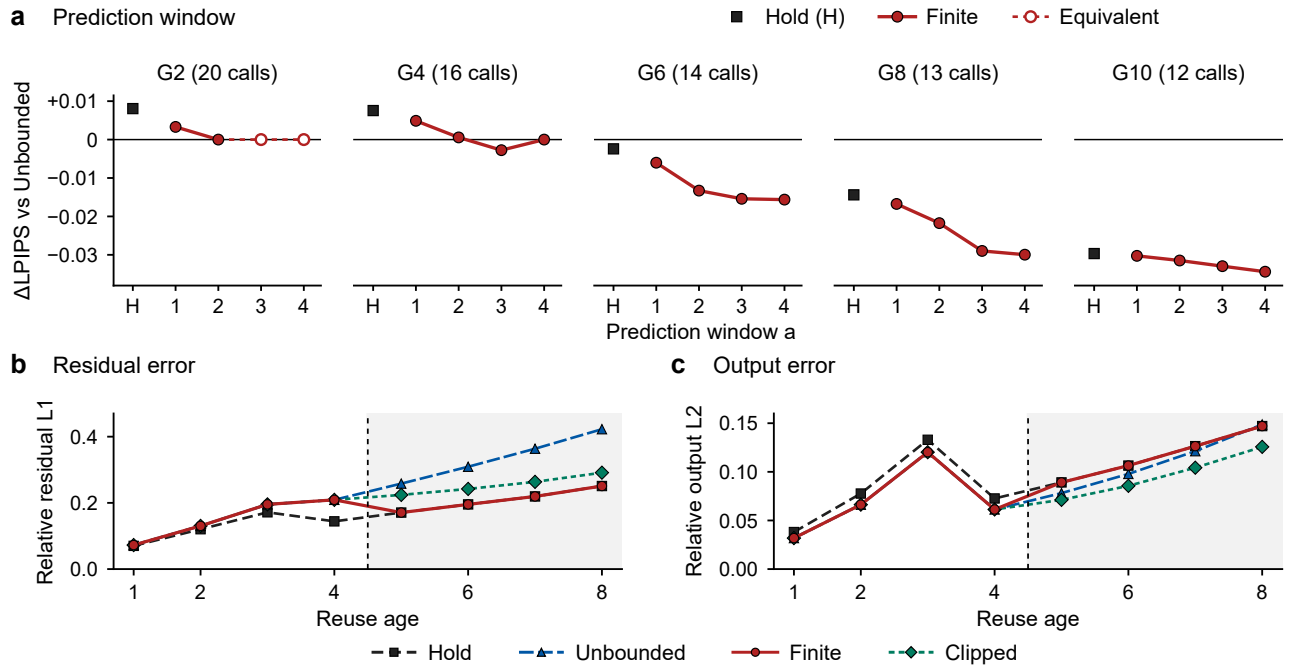


Figure 3. Forecast horizon and local approximation error. a, Development mean LPIPS relative to Unbounded, using 80 prompts and two seeds per refresh schedule. Panels share a scale. Negative values indicate lower LPIPS than Unbounded. Black squares (H) show Hold, filled red circles show measured finite windows, and the zero line denotes Unbounded. For $g2$, hollow red circles and dashed segments mark $a = 3, 4$ as analytic equivalents of $a = 2$ (Proposition 2). Parentheses in the titles give full-evaluation counts. **b,c,** Relative residual L1 and output L2 errors on the Hold trajectory under $g8$, weighted by observed step counts. Shading marks ages beyond $a = 4$. These descriptive means combine differences in age, phase and anchor spacing. Diagnostic time is excluded from speed measurements.

prescribed refresh steps.

Development selects $a = 3$ for $g4$ (16 full calls) and $a = 4$ for $g8$ (13 full calls). All six paired confirmation comparisons favor Finite (Table 2). Under $g8$, mean LPIPS is 29.2% lower than Unbounded and 15.1% lower than Clipped. Removing the two previously exposed prompts leaves all six confidence intervals below zero and all adjusted probabilities below 0.01.

Clipped shows that returning to an observation helps beyond stopping extrapolation at the window boundary. This benefit is an average effect. Finite has higher LPIPS than Clipped in 64 of 160 pairs under $g4$ and 12 under $g8$. Both refresh spacing and sampling phase change between schedules. Across development schedules (Fig. 3a), the best tested choice moves from equivalent $a \geq 2$ policies at $g2$ to $a = 3$ at $g4$ and $a = 4$ at longer gaps. Four is the largest finite window tested.

The retained development means also check Proposition 2. At $g2$, $a = 2$ and Unbounded have exactly equal exported LPIPS, SSIM and PSNR means. At $g4$, the same equality holds for $a = 4$. These equalities explain why a window ablation must include steps beyond the window.

Why local error and image fidelity can differ. At selected steps, a fresh model output lets us compare all four estimates at the same current state and anchors. We repeat this on trajectories generated by Hold and Unbounded, called *carriers*. The diagnostic covers 16 development prompts, two seeds, two schedules and two carriers, totaling 128 trajectories and 3,264 eligible reuse steps.

Under $g8$, Finite lowers relative residual L1 error but slightly increases relative output L2 error on both carriers. This observation is compatible with the geometry in Section 3.4, whose Euclidean residual criterion con-



Figure 4. Preserving windscreen details. A selected example (`img_0253`, seed 2) compares the reference with four caching methods. The top row shows complete images, with the enlarged region boxed. The bottom row compares the same windscreen crop at a common scale. LPIPS is measured on the complete images.

cerns a different norm and space. The head can weight residual directions differently, and later updates propagate local errors. Prediction is less accurate in the early recorded phase and more useful during much of the late phase. SARCache uses a fixed window, with phase adaptation left for future study.

Transfer and video diagnostics. Krea transfer uses the same $a = 3$ window on 40 prompts and two seeds. Under $g8$, Finite improves over Hold but is less accurate than Clipped and Unbounded. Clipped also has the lowest mean under $g4$. These descriptive results use rounded aggregate measurements. The prompt partition includes five fallback admissions and is not established as fully independent.

The corrected H3 development results favor Hold on both schedules. Replacing 223 records from an incompatible generation run restores the expected $g4$ equality, with all 80 paired LPIPS values for $a = 4$ and Unbounded coinciding. Both schedules use eight full evaluations. This correction concerns development data. The original system evaluation uses one seed and mixed media sources. Video system results remain exploratory pending a completed evaluation with multiple seeds.

4.4 Runtime and memory

The Qwen cost study uses 16 calibration prompts, two seeds and five repetitions per configuration and timing protocol (Table 3). SARCache reduces request time, with recorded cache operations accounting for about 1–1.5% of denoising time. This category includes work performed during reuse. Timing dispersion combines prompt, seed and repetition effects. Full and SARCache each report peak allocated memory of 36.86 GiB after rounding, which leaves the incremental cache cost unresolved. These measurements use the original A/B configurations. Reserved memory and isolated cache increments are unavailable.

Table 3. Qwen-1024 runtime. Mean request times measured at request boundaries and cache-operation fractions from separate instrumented runs. Each row and protocol has 160 observations across prompts, seeds and repetitions.

Configuration	Request (s)	Full calls	Ops / denoise
Full	6.270	40.0	0.06%
SARCache A	3.521	21.9	0.97%
SARCache B	2.473	15.0	1.47%
SeaCache A	3.276	20.1	0.10%
EasyCache A	3.252	20.2	0.67%

5 Discussion

The main finding is that reuse and prediction can end at different steps. Qwen benefits from predicting briefly and then returning to an observed residual. The comparison with Clipped supports the return itself. Proposition 1 explains the local trade-off through the alignment between the historical trend and the current residual displacement. Corollary 1 gives a conservative rationale when that direction is uncertain.

The best window depends on the model and schedule. A short reuse cap makes the return inactive, as Proposition 2 shows. Krea favors other policies in the tested transfer schedules. Changes in the model, solver, representation and schedule jointly affect these outcomes. The fixed window controls prediction duration, while the gate controls refresh timing online.

The experiments measure agreement with the uncached sampler. Comparisons with newer predictors

under a common protocol, broader transfer tests, and prompt-adherence and human-preference studies would clarify the method's scope. Rounded confirmation records support statistical reconstruction, while missing original images limit independent metric verification. Selected JPEG examples illustrate local behavior. Cleaner video data and additional seeds are needed to assess generality in joint generation.

6 Conclusion

SARCache separates residual reuse from extrapolation of recent changes. It predicts within a finite window, then returns to the latest observation while updating the current projection and head. The analysis identifies when this return reduces local residual error and when the refresh schedule makes the window inactive. Controlled Qwen experiments support the policy at identical full-evaluation budgets. System, transfer and diagnostic results show how its benefits depend on the model and sampling schedule.

LLM Usage Statement

LLMs assisted with writing, literature review, metadata organization, and analytical derivations.

References

[1] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising Diffusion Probabilistic Models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851. Curran Associates, Inc., 2020. URL `https://proceedings.neurips.cc/paper_files/paper/2020/file/4c5bcfec8584af0d967f1ab10179ca4b-Paper.pdf`.

[2] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-Based Generative Modeling through Stochastic Differential Equations. In *International Conference on Learning Representations*, 2021. URL `https://arxiv.org/abs/2011.13456`.

[3] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-Resolution Image Synthesis With Latent Diffusion Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10684–10695, June 2022. URL `https://openaccess.thecvf.com/content/CVPR2022/html/Rombach_High-Resolution_Image_Synthesis_With_Latent_Diffusion_Models_CVPR_2022_paper.html`.

[4] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4195–4205, October 2023.

[5] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, Dustin Podell, Tim Dockhorn, Zion English, and Robin Rombach. Scaling rectified flow transformers for high-resolution image synthesis. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 12606–12633. PMLR, 21–27 Jul 2024. URL `https://proceedings.mlr.press/v235/esser24a.html`.

[6] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022. doi: 10.48550/arXiv.2210.02747.

[7] Feng Liu, Shiwei Zhang, Xiaofeng Wang, Yujie Wei, Haonan Qiu, Yuzhong Zhao, Yingya Zhang, Qixiang Ye, and Fang Wan. Timestep embedding tells: It's time to cache for video diffusion model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7353–7363, June 2025.

[8] Xin Zhou, Dingkang Liang, Kaijin Chen, Tianrui Feng, Xiwu Chen, Hongkai Lin, Yikang Ding, Feiyang Tan, Hengshuang Zhao, and Xiang Bai. Less is enough: Training-free video diffusion acceleration via runtime-adaptive caching. *arXiv preprint arXiv:2507.02860*, 2025. doi: 10.48550/arXiv.2507.02860.

[9] Jiwoo Chung, Sangeek Hyun, MinKyu Lee, Byeongju Han, Geonho Cha, Dongyoon Wee, Youngjun Hong, and Jae-Pil Heo. SeaCache: Spectral-Evolution-Aware Cache for Accelerating Diffusion Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14283–14294, 2026.

[10] Jiacheng Liu, Chang Zou, Yuanhuiyi Lyu, Junjie Chen, and Linfeng Zhang. From reusing to forecasting: Accelerating diffusion models with taylorseers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 15853–15863, October 2025.

[11] Xiaoliu Guan, Lielin Jiang, Hanqi Chen, Xu Zhang, Jiaxing Yan, Guanzhong Wang, Yi Liu, Zetao Zhang, and Yu Wu. Forecasting When to Forecast: Accelerating Diffusion Models with Confidence-Gated Taylor. *arXiv preprint arXiv:2508.02240*, 2025.

[12] Jiazi Bu, Pengyang Ling, Yujie Zhou, Yibin Wang, Yuhang Zang, Dahua Lin, and Jiaqi Wang. DiCache: Let Diffusion Model Determine Its Own Cache. In *International Conference on Learning Representations*, 2026.

[13] Byunggwan Son, Jeimin Jeon, Jeongwoo Choi, and Bumsub Ham. Relational Feature Caching for Accelerating Diffusion Transformers. In *International Conference on Learning Representations*, 2026.

[14] Felix Wimbauer, Bichen Wu, Edgar Schoenfeld, Xiaoliang Dai, Ji Hou, Zijian He, Artsiom Sanakoyeu, Peizhao Zhang, Sam Tsai, Jonas Kohler, Christian Rupprecht, Daniel Cremers, Peter Vajda, and Jialiang Wang. Cache Me if You Can: Accelerating Diffusion Models through Block Caching. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6211–6220, 2024.

[15] Xinyin Ma, Gongfan Fang, and Xinchao Wang. DeepCache: Accelerating Diffusion Models for Free. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15762–15772, June 2024. URL https://openaccess.thecvf.com/content/CVPR2024/html/Ma_DeepCache_Accelerating_Diffusion_Models_for_Free_CVPR_2024_paper.html.

[16] Senmao Li, Taihang Hu, Joost van de Weijer, Fahad Shahbaz Khan, Tao Liu, Linxuan Li, Shiqi Yang, Yaxing Wang, Ming-Ming Cheng, and Jian Yang. Faster Diffusion: Rethinking the Role of the Encoder for Diffusion Model Inference. In *Advances in Neural Information Processing Systems*, volume 37, pages 85203–85240. Curran Associates, Inc., 2024. doi: 10.52202/079017-2706. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/9ad996b5c45130de2bc00b60d8607904-Paper-Conference.pdf.

[17] Pratheba Selvaraju, Tianyu Ding, Tianyi Chen, Ilya Zharkov, and Luming Liang. FORA: Fast-Forward Caching in Diffusion Transformer Acceleration. *arXiv preprint arXiv:2407.01425*, 2024. doi: 10.48550/arXiv.2407.01425. URL https://arxiv.org/abs/2407.01425.

[18] Xuanlei Zhao, Xiaolong Jin, Kai Wang, and Yang You. Real-Time Video Generation with Pyramid Attention Broadcast. In *International Conference on Learning Representations*, volume 2025, pages 3296–3319, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/092c2d45005ea2db40fc24c470663416-Paper-Conference.pdf.

[19] Chang Zou, Xuyang Liu, Ting Liu, Siteng Huang, and Linfeng Zhang. Accelerating Diffusion Transformers with Token-wise Feature Caching. In *International Conference on Learning Representations*, volume 2025, pages 75467–75488, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/bbe024e0517fe12ac3a8d388b19ff9fe-Paper-Conference.pdf.

[20] Zhengyao Lyu, Chenyang Si, Junhao Song, Zhenyu Yang, Yu Qiao, Ziwei Liu, and Kwan-Yee K Wong. FasterCache: Training-Free Video Diffusion Model Acceleration with High Quality. In *International Conference on Learning Representations*, volume 2025, pages 33132–33156, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/518046d86bbc41a0707727c38301ad8e-Paper-Conference.pdf.

[21] Pengtao Chen, Mingzhu Shen, Peng Ye, Jianjian Cao, Chongjun Tu, Christos-Savvas Bouganis, Yiren Zhao, and Tao Chen. δ-DiT: A training-free acceleration method tailored for diffusion transformers. *arXiv preprint arXiv:2406.01125*, 2024. doi: 10.48550/arXiv.2406.01125. URL https://arxiv.org/abs/2406.01125.

[22] Zehong Ma, Longhui Wei, Feng Wang, Shiliang Zhang, and Qi Tian. MagCache: Fast Video Generation with Magnitude-Aware Cache. In *Advances in Neural Information Processing Systems*, volume 38, 2025. doi: 10.52202/085713-1153.

[23] ParaAttention Contributors. ParaAttention: First block cache. Software, 2025. URL https://github.com/chengzeyi/ParaAttention.

[24] Shikang Zheng, Liang Feng, Xinyu Wang, Qinming Zhou, Peiliang Cai, Chang Zou, Jiacheng Liu, Yuqi Lin, Junjie Chen, Yue Ma, and Linfeng Zhang. Forecast Then Calibrate: Feature Caching as ODE for Efficient Diffusion Transformers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 13449–13457, 2026. doi: 10.1609/aaai.v40i16.38349.

[25] Liang Feng, Shikang Zheng, Jiacheng Liu, Yuqi Lin, Qinming Zhou, Peiliang Cai, Xinyu Wang, Junjie Chen, Chang Zou, Yue Ma, and Linfeng Zhang. HiCache: A Plug-in Scaled-Hermite Upgrade for Taylor-Style Cache-then-Forecast Diffusion Acceleration. *arXiv preprint arXiv:2508.16984*, 2025.

[26] Enhuai Liu, Yunke Wang, Changming Sun, and Chang Xu. D2Cache: Second-Order Delta Caching for Higher Video Diffusion Acceleration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 43589–43599, 2026.

[27] Zhirong Shen, Rui Huang, Jiacheng Liu, Chang Zou, Peiliang Cai, Shikang Zheng, Zhengyi Shi, Liang Feng, and Linfeng Zhang. Beyond Fixed Formulas: Data-Driven Linear Predictor for Efficient Diffusion Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 30792–30801, 2026.

[28] Shikang Zheng, Guantao Chen, Qinming Zhou, Yuqi Lin, Lixuan He, Chang Zou, Peiliang Cai, Jiacheng Liu, and Linfeng Zhang. Let Features Decide Their Own Solvers: Hybrid Feature Caching for Diffusion Transformers. In *International Conference on Learning Representations*, 2026.

[29] Hugging Face Diffusers Contributors. Diffusers cache hooks. Software, 2026. URL https://github.com/huggingface/diffusers. Project-pinned source commit bdc2bea37a36038c44452811610489ea30ede229.

[30] Kerry Xu, Param Misra, Zachary Stenglein, Athri Karunamurthy, Fanjiang Ye, and Yuke Wang. Frequency-Step Cache: Separating Synchronization and Repair for Efficient Autoregressive Video Diffusion. *arXiv preprint arXiv:2610.15922*, 2026.

[31] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising Diffusion Implicit Models. In *International Conference on Learning Representations*, 2021. URL https://arxiv.org/abs/2010.02502.

[32] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the Design Space of Diffusion-Based Generative Models. In *Advances in Neural Information Processing Systems*, volume 35, pages 26565–26577. Curran Associates, Inc., 2022. doi: 10.52202/068431-1926. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/a98846e9d9cc01cfb87eb694d946ce6b-Paper-Conference.pdf.

[33] Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Model Sampling in Around 10 Steps. In *Advances in Neural Information Processing Systems*, volume 35, pages 5775–5787. Curran Associates, Inc., 2022. doi: 10.52202/068431-0418. URL https://proceedings.neurips.cc/paper_files/pap

er/2022/file/260a14acce2a89dad36adc8eefe7c59e-Paper-Conference.pdf.

[34] Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. DPM-Solver++: Fast Solver for Guided Sampling of Diffusion Probabilistic Models. *Machine Intelligence Research*, 22(4):730–751, 2025. doi: 10.1007/s11633-025-1562-4. URL https://link.springer.com/article/10.1007/s11633-025-1562-4.

[35] Wenliang Zhao, Lujia Bai, Yongming Rao, Jie Zhou, and Jiwen Lu. UniPC: A Unified Predictor-Corrector Framework for Fast Sampling of Diffusion Models. In *Advances in Neural Information Processing Systems*, volume 36, pages 49842–49869. Curran Associates, Inc., 2023. doi: 10.52202/075280-2170. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/9c2aa1e456ea543997f6927295196381-Paper-Conference.pdf.

[36] Tim Salimans and Jonathan Ho. Progressive Distillation for Fast Sampling of Diffusion Models. In *International Conference on Learning Representations*, 2022. URL https://arxiv.org/abs/2202.00512.

[37] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency Models. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 32211–32252. PMLR, 23–29 Jul 2023. URL https://proceedings.mlr.press/v202/song23a.html.

[38] Simian Luo, Yiqin Tan, Longbo Huang, Jian Li, and Hang Zhao. Latent Consistency Models: Synthesizing High-Resolution Images with Few-Step Inference. *arXiv preprint arXiv:2310.04378*, 2023. doi: 10.48550/arXiv.2310.04378. URL https://arxiv.org/abs/2310.04378.

[39] Muyang Li, Tianle Cai, Jiaxin Cao, Qinsheng Zhang, Han Cai, Junjie Bai, Yangqing Jia, Kai Li, and Song Han. DistriFusion: Distributed Parallel Inference for High-Resolution Diffusion Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7183–7193, June 2024. URL https://openaccess.thecvf.com/content/CVPR2024/html/Li_DistriFusion_Distributed_Parallel_Inference_for_High-Resolution_Diffusion_Models_CVPR_2024_paper.html.

[40] Zigeng Chen, Xinyin Ma, Gongfan Fang, Zhenxiong Tan, and Xinchao Wang. AsyncDiff: Parallelizing Diffusion Models by Asynchronous Denoising. In *Advances in Neural Information Processing Systems*, volume 37, pages 95170–95197. Curran Associates, Inc., 2024. doi: 10.52202/079017-3015. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/ad15848baa3932c0d2deabf0e11d1dcd-Paper-Conference.pdf.

[41] Chitwan Saharia, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily Denton, Seyed Kamyar Seyed Ghasemipour, Burcu Karagol Ayan, S. Sara Mahdavi, Rapha Gontijo Lopes, Tim Salimans, Jonathan Ho, David J Fleet, and Mohammad Norouzi. Photorealistic text-to-image diffusion models with deep language understanding. *arXiv preprint arXiv:2205.11487*, 2022. doi: 10.48550/arXiv.2205.11487.

[42] Qwen Team. Qwen-Image-2.1: Compact, efficient, and unified image creation. Official model release, 2026. URL https://qwen.ai/blog?id=qwen-image-2.1.

[43] Krea AI. Krea 2 Raw: Model card and official inference code. Model and software release, 2026. URL https://huggingface.co/krea/Krea-2-Raw. Accessed October 9, 2026.

[44] MiniMax. MiniMax-H3: Model card. Model repository, 2026. URL https://huggingface.co/MiniMaxAI/MiniMax-H3. Accessed October 10, 2026.

[45] Comfy Org. MiniMax H3: Native video generation workflows. Software documentation, 2026. URL https://docs.comfy.org/tutorials/video/minimax/minimax-h3-native. Accessed October 10, 2026.

[46] Google Research. PartiPrompts: Prompt dataset and source repository. Dataset repository. URL https://github.com/google-research/parti. Accessed October 14, 2026.

[47] GenEval Contributors. GenEval: Prompt data and evaluation repository. Dataset and software repository. URL https://github.com/djghosh13/geneval. Accessed October 14, 2026.

[48] VBench Contributors. VBench: Video generation evaluation and prompt repository. Dataset and software repository. URL https://github.com/Vchitect/VBench. Prompt subsets from the revision recorded in the experiment manifest. Accessed October 14, 2026.

[49] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018. URL https://richzhang.github.io/PerceptualSimilarity/.

[50] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, April 2004. URL https://www.cns.nyu.edu/~lcv/ssim/.